

MODULE 07

Skills、Hooks、MCP、Plugins 与 LSP

四层扩展模型：上下文、生命周期、外部工具、可分发能力包

扩展机制不是越多越好。Skill 改变模型知道什么，Hook 改变事件发生时必做什么，MCP 增加外部工具，Plugin 把它们封装和分发。

研究基线：Claude Code v2.1.236（稳定版） | Python Agent SDK v0.2.141（tag） | 历史源码观察 v2.1.88
资料核验日期：2026-08-19 | 中文深度研究版

目录

1. 先用决策表选机制
2. Skills：目录就是可执行知识包
3. Hooks：把概率行为变成确定行为
4. Hook I/O 与决策语义
5. MCP：外部能力的标准边界
6. SDK MCP Bridge 源码
7. Plugins：可分发组合层
8. LSP 与代码智能
9. 供应链与版本治理
10. 一个可维护插件的验收标准

阅读提示：内部模块名以历史源码观察解释，当前功能以官方文档与 v0.2.141 SDK 源码校验。

1. 先用决策表选机制

需求	首选	原因	上下文成本
可复用领域 workflow	Skill	按需注入说明和资源	低
每次编辑后格式化	Hook	确定性生命周期动作	近零/仅结果
访问 Jira/DB/API	MCP	标准工具与资源协议	中到高
团队安装整套能力	Plugin	打包 skill/agent/hook/MCP/LSP	取决于组件
符号定义/引用/诊断	LSP plugin	结构化代码智能	按调用
隔离研究任务	Subagent	新 context window	高

2. Skills：目录就是可执行知识包

Claude Code 遵循 Agent Skills 标准，并扩展 invocation control、subagent execution 和 dynamic context injection。`.claude/commands/foo.md` 与 `.claude/skills/foo/SKILL.md` 已统一为相近调用体验，但 skill 目录更适合带 references、scripts 和 assets。 [O16]

字段	作用
name / description	发现与模型选择；description 决定何时触发
disable-model-invocation	只允许用户显式调用
user-invocable	是否出现在用户命令面
allowed-tools / allowedTools	声明/预批准技能所需工具
context	inline 或在 subagent 中执行
hooks	技能激活期间注册生命周期 hook

示例 | 最小可审查 skill

```
---
name: api-migration
description: Plan and verify backward-compatible API migrations
disable-model-invocation: true
---
# Workflow
1. Inventory callers.
2. Define compatibility contract.
3. Implement behind tests.
4. Run acceptance and rollback checks.
```

安全事实

Skill allowlist 是上下文过滤，不是文件隔离。不要把 token、账号或内部秘密写进 skill。 [S5]

3. Hooks：把概率行为变成确定行为

阶段	代表事件	典型用途
会话	SessionStart / SessionEnd / ConfigChange	注入环境、审计配置
输入	UserPromptSubmit	验证需求、附加上下文
工具前	PreToolUse / PermissionRequest	阻断、批准、修改 input
工具后	PostToolUse / PostToolUseFailure	格式化、测试、分类失败
Agent	SubagentStart / SubagentStop / Teammateldle	治理委托生命周期
Context	PreCompact / InstructionsLoaded	保存关键状态、审计加载
完成	Stop / TaskCompleted	质量门禁与通知

官方 hooks 支持 command、HTTP、MCP tool、prompt 和 agent handler。SDK hooks 在 initialize 时被编译成 callback IDs；CLI 触发时发送 `hook_callback` control request。多个同事件 matcher 可能并发执行，hook 不能依赖注册顺序。 [O17, S2, S5]

示例 | 确定性格式化 hook

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{"type": "command", "command": "./scripts/format-changed.sh"}]
    }]
  }
}
```

4. Hook I/O 与决策语义

- 输入是 JSON，包含 session_id、cwd、hook_event_name 以及事件特有字段。
- command hook 用 exit code + stdout JSON/文本表达 allow/block/context；不要把日志混进协议 stdout。
- async hook 适合通知，不适合决定是否执行当前工具。
- prompt/agent hook 会产生模型成本与延迟，应给 timeout 和 fallback。
- Post hook 发生在副作用之后，最多补救/告警，不能声称已撤销。
- Stop hook 若要求继续，必须有防递归条件，避免永不结束。 [O17]

5. MCP：外部能力的标准边界

MCP server 向 Claude Code 暴露 tools、resources 和 prompts。CLI 可通过 stdio、HTTP/SSE 及 claude.ai connector 等方式连接；SDK 还支持 in-process server。安装 scope 与 settings scope 决定 server 对一个项目、用户或组织是否可见。 [O18, O23]



风险	控制
工具目录过大	deferred loading / 只启用所需 server
server 过权	managed allowlist + scoped permission rules
OAuth/token 泄露	凭据存储与最小 scope，不回注 context
结果注入	把 server 文本当外部数据，保留来源
连接挂起	初始化 timeout、status/reconnect/toggle
协议版本漂移	能力协商、未知字段透传/忽略

6. SDK MCP Bridge 源码

Python SDK 的 `SdkMcpBridge` 为每个 in-process server 建立内存传输 session，处理 JSON-RPC request/response/notification；pending response 以 ID 关联，关闭时失败所有未完成请求。v0.2.140 起支持 MCP 1.x/2.x，并使用 MCP 自己的 in-memory transport，资源、prompts 和多种 result content 能完整到达 CLI。
[S1, S9]

CLI → SDK 的 `mcp_message` control request 由 bridge 处理后返回 `mcp_response`。这意味着 SDK 应用可以提供工具而不开放网络端口，但其 handler 仍运行在宿主进程权限内，必须单独做输入验证与授权。 [S2, S9]

7. Plugins：可分发组合层

组件	典型路径/字段
Manifest	.claude-plugin/plugin.json
Skills/commands	skills/*/SKILL.md, commands/*.md
Agents	agents/*.md
Hooks	hooks/hooks.json + handlers
MCP	.mcp.json / manifest config
LSP	.lsp.json 或 plugin.json inline
Settings/marketplace	plugin config + marketplace entry

官方 [anthropics/claude-code](#) 仓库提供 code-review、feature-dev、frontend-design、hookify、plugin-dev、security-guidance 等插件，可用于研究真实目录与组合模式。核心价值在“封装发布与命名空间”，不是引入另一套执行引擎。 [O19, O28]

8. LSP 与代码智能

Plugin 可在 `.lsp.json` 声明 language server command、args 和 extensionToLanguage。Claude 通过 LSP 工具获取 definition、references、diagnostics 等结构化结果，比 Grep 更了解语言语义。 [O19, O3]

- LSP server 是长期子进程，需要启动、重启、超时和日志治理。
- 路径必须映射到当前 worktree/cwd；gitignored file 是否返回应有明确策略。
- 诊断是提示，不是事实；仍需测试/编译验证。
- 大型 monorepo 应限制 workspace roots，避免 server 索引拖慢启动。

9. 供应链与版本治理

- Plugin marketplace 需要来源 allowlist、版本 pin、校验与撤销。
- 安装前显示将获得的 hooks、MCP、LSP 和脚本权限，而不只显示名称。
- project plugin 是仓库代码的一部分，workspace trust 必须先于执行。
- Skill/agent prompt 更新也能改变行为，应像代码 diff 一样审查。
- hook handler stdout 是协议面，增加 schema tests 与 timeout tests。
- MCP tool 名、schema 和结果 shape 变更应做 contract tests。

10. 一个可维护插件的验收标准

维度	验收
触发	Skill/agent description 在正例触发、负例不误触发
安全	最小工具集；危险动作仍 ask/deny
确定性	格式化/检查由 hook，不依赖模型记住
成本	不加载无关 MCP schemas；大资源按需
兼容	CLI 最低版本与平台依赖写清
恢复	handler 超时/失败不会锁死 session
可观测	关键 hook/MCP/tool 事件可追踪

参考资料与证据等级

[O16] 官方 | [Skills](#)

<https://code.claude.com/docs/en/skills>

[O17] 官方 | [Hooks reference](#)

<https://code.claude.com/docs/en/hooks>

[O18] 官方 | [MCP](#)

<https://code.claude.com/docs/en/mcp>

[O19] 官方 | [Plugins reference](#)

<https://code.claude.com/docs/en/plugins-reference>

[O23] 官方 | [Agent SDK overview](#)

<https://code.claude.com/docs/en/agent-sdk/overview>

[S2] 开源源码 | [Internal Query control protocol](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/query.py

[S5] 开源源码 | [SDK types and options](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/types.py

[S9] 开源源码 | [In-process SDK MCP bridge](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/sdk_mcp_bridge.py

[O28] 官方 | [GitHub repository / public examples](#)

<https://github.com/anthropics/claude-code>

方法说明

官方材料用于确认当前稳定行为；SDK 源码固定在发布 tag v0.2.141；社区材料只用于解释 v2.1.88 历史源码结构。源码行号和内部函数名可能在后续版本变化，外部契约以官方最新文档为准。